

Полная переписка

Экспорт пользовательской переписки в PDF без сокращений.

ПЕРЕПИСКА

Пользователь:

Как лучше всего реализовывать это всё?

где нужно", а bounded latency применять как policy на уровне бюджета, а не как отдельный второй алгоритм. Для compare это означает fast path для структурного сопоставления и deep/semantic path только при слабой уверенности, heterogeneous-парах или когда обычный redline уже не даёт полезного вывода. Для single-document analysis текущая реализация уже ближе к правильной модели, чем compare: там bounded latency встроен в reduce-политику, а не размазан по логике. можешь это на более русском языке написать, потому что некоторые понятия мне слабо понятны на английском?

- Да. Вот та же мысль, но по-русски и без англицизмов.

Для анализа одного документа я бы оставил текущий подход: сначала делаем дешёвые и предсказуемые шаги, которые почти не зависят от модели, а к LLM обращаемся только там, где без неё уже нельзя. Ограничение по времени и ресурсам лучше задавать не как отдельный "режим", а как политику бюджета: сколько текста можно прогнать, сколько шагов суммаризации позволить, когда можно сократить глубину обработки. только если документная пара сложная, уверенность низкая или обычное сравнение не даёт нормального вывода. То есть сначала "сравнили по форме и структуре", а потом, если нужно, "докопались до смысла".

Если совсем просто:

- bounded latency = верхняя граница на время и ресурсы, чтобы система не разрасталась в долгий и дорогой прогон.

Итоговый смысл такой: для одного документа у нас уже почти правильная схема, а для сравнения двух документов стоит сильнее разделить быстрый структурный этап и более глубокий смысловой этап.

анализу документа и ответу по нему?

документу: извлечение фактов, опора на источник, цитирование, и когда модель должна молчать вместо фантазирования.

- Searched official documentation grounded answers citations document question answering best practices
- Searched site:platform.openai.com file search citations source passages document answer
- Searching the web
- Searched arXiv retrieval augmented generation for knowledge-intensive NLP tasks abstract
- Searching the web
- Searched <https://help.openai.com/en/articles/8313428-does-chatgpt-tell-the-truth%3F.pls>
- Searching the web
- Searched 'retrieve information in a knowledge base' in <https://platform.openai.com/docs/guides/tools-file-search/>

-
- В интернете по этой теме обычно сходятся не на "красивом ответе", а на схеме найти опору в источнике → собрать факты → сформулировать вывод → показать, откуда он взят.

Самые частые мысли такие:

- Не отвечать “из головы”. Для документов модель должна опираться на сам текст или подключённые источники, потому что без этого она может уверенно ошибаться или додумывать.
- Сначала извлечение, потом ответ. Хорошая схема: сначала вытащить релевантные фрагменты, потом уже из них собирать краткий вывод.
- Нужны ссылки на источник. Для ответов по документам почти всегда советуют показывать, на чём основан вывод: страницы, абзацы, цитаты, passages, ссылки.
- Если доказательств нет, надо так и сказать. Нормальная практика: “в документе это не найдено”, а не пытаться заполнить пробел.
- Для сложных документов нужен многошаговый разбор. Сначала общий обзор, потом детализация по пунктам, потом финальная сводка.
- Для длинных документов полезно разделять ответ на summary и appendix. Это снижает шум: наверху короткий вывод, ниже подробности.
- Если вопрос аналитический, а не фактологический, нужен план и контроль источников. То есть не просто искать совпадения слов, а явно понимать, что именно сравнивается и почему.
- OpenAI Help: ChatGPT can be helpful, but it's not always right
- OpenAI API: File search (<https://platform.openai.com/docs/guides/tools-file-search/>)
- Patrick Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (<https://www.patricklewis.io/publication/rag/>)

› В нашем TASKS.md описаны эти задачи. Просто для ускорения инференса нельзя ли ускорить работу по ответу на документ? То есть разделение анализа документа на fast и deep. И как это реализовать в нашем текущем контексте с вектором направления на развитие как mcp и выбора инструмента как быструю обработку и глубокую

└ Read SKILL.md
Search TASKS.md in .

• Сейчас подниму TASKS.md, чтобы связать ответ с уже поставленными задачами и не предлагать лишнюю архитектуру. Потом сведу это к двум-трём реалистичным вариантам в вашем текущем стеке.

• Explored
└ Read TASKS.md
Search fast|deep|analysis|document|compare|mcp|tool in TASKS.md

• Да. И в вашем контексте это лучше делать не как “два режима одного и того же длинного промпта”, а как маршрутизацию между разными инструментами.

Самая практичная схема такая:

- Fast: быстро найти нужные фрагменты, извлечь факты, ответить коротко и с опорой на источник.
- Deep: если вопрос сложный, противоречивый или требует общего вывода по документу, тогда запускать более тяжёлую суммаризацию/синтез.

То есть не “сначала всегда глубокий анализ, потом сокращаем”, а “сначала дешёвый поиск ответа, и только если этого недостаточно — глубокая обработка”.

В вашем коде это уже частично видно:

- document_analysis уже устроен как линейный конвейер с budget-aware reduce-политикой, а не как один монолитный вызов backend/orchestrator/workflows/document_analysis.py:806.
 - Политика fast/deep для суммаризации уже живёт отдельно в backend/orchestrator/summary_reduce_policy.py:1.
 - На уровне оркестрации у вас уже есть разведение по интендам: document_analysis, compare_documents, document_question backend/orchestrator/orchestration_runtime.py:143.
- Если переводить это на нормальный язык, я бы предложил такой контракт:

- `document_question = fast path`
Быстрое извлечение релевантных фрагментов, ответ с цитатами, минимум лишней генерации.
- `document_analysis = deep path`
Быстрое структурное сравнение, а глубокий смысловой вывод только при необходимости
`backend/orchestrator/workflows/compare.py:618`.

Что делать практически:

- Добавить явный `answer_mode` или `analysis_depth` в `runtime policy`.
- Сначала пытаться ответить через быстрый путь: загрузка, разбиение, `retrieval`, `extraction`.
- Если уверенность низкая, источников мало, есть конфликт фактов или вопрос просит “в целом проанализировать”, поднимать `deep path`.
- Для ответов по документу держать обязательные цитаты и “не знаю”, если опоры нет. Это важнее, чем любой красивый длинный ответ.

Если хочешь, я могу дальше предложить уже конкретную схему для вашего `TASKS.md`: какие 2-3 задачи туда добавить, чтобы `fast/deep` для документов легли на текущую архитектуру без лишнего редизайна.

> `document_question` по сути будет работать как RAG, как и сейчас работает в принципе, нет?
Если сам анализ документа `document_analysis` разбивать на глубокий анализ или быстрый анализ. Аналогично разделять и по сравнению документов: (`compare_documents`) на глубокий анализ или быстрый анализ

- Да, именно так я бы это и разложил.

`document_question` лучше оставить как RAG-ответ по документам: достать релевантные фрагменты, проверить опору в тексте, ответить с цитатами. Это не “анализ документа” в полном смысле, а режим вопрос → поиск → ответ. У него свои параметры глубины поиска, но это отдельная история.

А вот `document_analysis` и `compare_documents` как раз и стоит делить на `fast` и `deep`:

- `document_analysis / fast`:
 - быстро вытащить ключевые факты;
 - сделать короткую сводку;
 - не идти в полный многошаговый синтез, если задача простая.
- `document_analysis / deep`:
 - полный разбор;
 - больше проходов по тексту;
 - более широкая сводка, риски, детали, структура.
- `compare_documents / fast`:
 - структурное сравнение;
 - показать основные различия;
 - не запускать тяжёлый смысловой анализ, если достаточно `redline`-подобного вывода.
- `compare_documents / deep`:
 - смысловое сравнение;
 - отношения документов;
 - пробелы покрытия;
 - конфликты;
 - рекомендации.

То есть правильнее разделять не по линии `question vs analysis`, а по двум осям:

- тип задачи: `document_question`, `document_analysis`, `compare_documents`;
- как RAG-путь.

Если хочешь, я могу следующим сообщением предложить уже конкретный контракт: какие поля добавить в `runtime/state`, как назвать режимы, и по каким триггерам переключать `fast/deep` в каждом из двух `workflow`.

- Model changed to gpt-5.4 medium
В том числе вопрос выбора этих инструментов. В будущем это будут реальные mcp инструменты в open web ui, которые можно задавать через slash. Тем самым мы уйдём от оркестратора как такового, в будущем его нужно будет либо дообучать под наши задачи через LoRA (+ сделать его модульным, отключать по необходимости и вручную выбирать граф решений через mcp), либо же убрать его вовсе. Сформируй отдельный план

и желательно с понятными русскими терминами, в скобках можешь указать на английском по необходимости.

- Сначала посмотрю короткие инструкции для планирования и затем соберу отдельный план под ваш вектор: разделённые инструменты, выбор инструмента, переход к MCP/slash и постепенный уход от текущего оркестратора.

- Explored
 - └ Read SKILL.md

• # Plan

Собрать новую архитектуру документных инструментов вокруг явно разделённых сценариев: быстрый режим (fast) и глубокий режим (deep) для document_analysis и compare_documents, при этом оставить document_question как отдельный путь вопрос-ответ по документам на базе поиска по фрагментам (RAG). Переход делать поэтапно: сначала разделить контракты и выбор инструмента внутри текущего backend, затем вынести это в реальные MCP-инструменты со slash-вызовом в Open WebUI, чтобы оркестратор стал либо тонким совместимым слоем, либо опциональным модулем.

Scope

- In:
 - Разделение document_analysis и compare_documents на fast / deep
 - Явный контракт выбора инструмента и режима выполнения
 - Переходный слой от текущего оркестратора к MCP-tools / slash-командам
 - План по модульности оркестратора и его возможному отключению
 - Терминология, telemetry и критерии выбора режима
- Out:
 - Немедленная реализация LoRA-дообучения
 - Полный отказ от оркестратора в одном шаге
 - UI-детали Open WebUI beyond tool contract
 - Тонкая настройка моделей и prompt-тюнинг на этапе архитектурного разделения

Action items

[] Зафиксировать целевую матрицу инструментов: document_question, document_analysis_fast, document_analysis_deep, compare_documents_fast, compare_documents_deep, documents_summary, и описать для каждого вход, выход, допустимые файлы, тип ответа и требования к цитированию в TASKS.md и отдельном design/список документов.

[] Разделить на уровне backend не только intent, но и execution mode: добавить явное поле выбора режима (analysis_depth / tool_variant) в контракты orchestration_runtime.py, execution_runtime.py и session-state, чтобы режим был виден, сохранялся и мог задаваться вручную без эвристики.

[] Переделать document_analysis в два самостоятельных исполняемых пути вместо одного workflow с неявной деградацией: быстрый путь должен делать извлечение ключевых фактов и короткую сводку, глубокий путь должен включать полный многошаговый синтез, расширенные секции отчёта и более дорогую обработку backend/orchestrator/workflows/document_analysis.py.

[] Переделать compare_documents в два самостоятельных пути: быстрый путь должен выполнять структурное сравнение и краткий отчёт по основным различиям, глубокий путь должен добавлять смысловой анализ, покрытие тем, конфликты и рекомендации; отдельно исправить известные backlog-проблемы с LLM-очередью и single-item JSON parsing из R1.0.10–R1.0.12 в TASKS.md.

[] Вынести выбор инструмента в отдельный модуль политики (tool selection policy): он должен принимать запрос, число документов, тип документов, явный slash-вызов, ручной override и runtime budget, а возвращать не “намерение вообще”, а конкретный инструмент и режим; вероятные точки интеграции: backend/orchestrator/orchestration_runtime.py и новый backend module рядом с tool-routing.

[] Ввести два способа выбора инструмента: автоматический (auto tool selection) и ручной

(manual tool selection). Автоматический нужен для совместимости с текущим чатом, ручной нужен как целевой продуктовый путь через slash-команды в Open WebUI, например /doc-fast, /doc-deep, /compare-fast, /compare-deep, /ask-doc.

[] Спроектировать MCP-слой как канонический интерфейс будущего: каждый инструмент должен стать отдельным MCP tool с узким контрактом, а текущий оркестратор должен перейти в статус compatibility-layer, который либо вызывает те же MCP-инструменты, либо полностью отключается feature-flag'ом / config-опцией.

[] Подготовить стратегию ухода от оркестратора в два этапа: сначала сделать его модульным и отключаемым (orchestrator_mode=enabled|compat|off), затем сравнить два дальнейших направления – LoRA-усиление для роутинга и выбора графа против полного перехода на явный MCP/manual invocation – и принять решение по данным telemetry, а не заранее.

[] Добавить telemetry и quality-контракт именно на уровне инструмента и режима: нужно измерять latency, число LLM-вызовов, долю fallback, число найденных цитат, полноту покрытия и долю ручных переопределений, чтобы потом честно сравнить fast против deep и автоматический выбор против ручного backend/orchestrator/telemetry_runtime.py.

[] Запланировать валидацию по трём срезам: unit tests на policy выбора инструмента, workflow tests на fast/deep-ветки document_analysis и compare_documents, и интеграционный smoke для slash/MCP-контракта в Open WebUI; дополнительно нужен benchmark-срез по времени и числу вызовов модели для offline bundle и обычного runtime.

Open questions

- Нужен ли в продукте дефолтный автоматический выбор инструмента после перехода на slash/MCP, или ручной выбор должен стать основным режимом, а auto остаться только как совместимость?
- Должны ли fast и deep быть отдельными инструментами с разными именами, или это один инструмент с параметром режима?
- Хотите ли вы вести переход через “оркестратор вызывает MCP tools”, или предпочитаете быстрее строить целевую схему, где UI вызывает MCP tools напрямую, а оркестратор постепенно выключается?``

Пользователь:

ещё же есть equipment . Его также касаться должно.

Ассистент:

Да, `equipment` тоже обязательно нужно включать в эту схему. Более того, у вас он по смыслу стоит между обычным анализом документа и сравнением документов: это уже не просто “вопрос-ответ по тексту”, но ещё и не классический redline. В `equipment.py` у вас отдельный workflow: `extract -> route -> match -> evaluate -> report`, с двухпроходной экстракцией, семантическим матчингом и пакетной LLM-оценкой. То есть это уже самостоятельный специализированный инструмент, а не частный случай `document\_analysis` или `compare\_documents`.

Тогда целевая картина получается не из 5 путей, а из 7:

- `ask\_document`
- `analyze\_document\_fast`
- `analyze\_document\_deep`
- `compare\_documents\_fast`
- `compare\_documents\_deep`
- `analyze\_equipment\_fast`
- `analyze\_equipment\_deep`

Это хорошо совпадает и с вашим runtime-слоем: в `orchestration\_runtime.py` `equipment\_analysis` уже выделен как отдельный route/executor, а в `execution\_runtime.py` он импортируется и участвует в общем выборе сценария. Значит, `equipment` не надо “впихивать” в compare или doc analysis – его лучше развивать как отдельную ветку со своими fast/deep-вариантами.

Почему это особенно важно для будущего MCP: по официальной спецификации MCP инструмент – это отдельно именованная операция с понятной схемой аргументов, а клиент сам решает, показывать её автоматически, через UI или через ручной вызов. То есть ваш план со slash-командами и явными специализированными инструментами полностью в духе MCP, и `equipment` туда ложится естественно как отдельный tool, а не как скрытая ветка оркестратора.

Как я бы разложил именно `equipment`.

`analyze\_equipment\_fast` – быстрый режим проверки соответствия.

Он должен делать только то, что даёт максимум пользы за минимальное время:

- извлечь позиции;
- определить режим `tz\_vs\_smata` или `smata\_vs\_smata`;
- сделать матчинг;
- выдать короткий отчёт по основным совпадениям, несовпадениям и отсутствующим позициям;
- LLM использовать только там, где без неё нельзя вынести вердикт по нескольким спорным позициям.

Это хорошо соответствует текущему pipeline: у вас уже есть deterministic/extraction-часть, затем matching, и только потом LLM evaluation батчами. Значит, fast-режим можно получить не переписыванием с нуля, а ужесточением лимитов и сокращением доли LLM на этапе `evaluate`.

`analyze\_equipment\_deep` – глубокий режим проверки закупочного соответствия.

Он должен добавлять:

- более глубокую нормализацию позиций;
- обработку неоднозначных текстовых требований;
- больше LLM-оценки на пограничных совпадениях;
- объяснение, почему позиция соответствует или не соответствует;
- сводку по рискам, пробелам и спорным местам.

Иными словами, deep в `equipment` – это не просто “тот же отчёт, но длиннее”, а режим, где система пытается понять смысл технического соответствия, а не только близость формулировок. Это особенно важно, потому что ваш equipment-workflow уже использует семантический матчинг через embeddings + Hungarian matching, а затем отдельную смысловую оценку батчами.

Я бы описал всю систему так, совсем по-русски:

- вопрос по документу – найти ответ в тексте;
- быстрый анализ документа – быстро понять, что в документе главное;
- глубокий анализ документа – сделать полноценный разбор;
- быстрое сравнение документов – показать основные различия;
- глубокое сравнение документов – понять смысловые расхождения;
- быстрая проверка оборудования – быстро проверить соответствие позиций;
- глубокая проверка оборудования – разобрать соответствие по смыслу и рискам.

С точки зрения кода я бы не делал отдельную архитектуру только для `equipment`. Лучше ввести общий каркас:

- `route` – тип задачи;
- `depth` – глубина;
- `selection\_mode` – кто выбрал: система или пользователь.

Например:

```
{
"route": "equipment_analysis",
"depth": "fast",
"selection_mode": "manual"
}
```

или

```
{
"route": "equipment_analysis",
"depth": "deep",
"selection_mode": "auto"
}
```

Это особенно логично у вас, потому что сейчас `document\_analysis` уже живёт с адаптивной reduce-политикой, `compare` уже фактически имеет structural vs semantic уровни, а `equipment` уже состоит из отдельных стадий extract/match/evaluate/report. То есть для всех трёх задач глубина может задаваться одним и тем же системным механизмом, но применяться по-разному внутри каждого workflow. Практически я бы сделал так.

Для ``equipment_fast``:

- уменьшить число LLM-проходов;
- строже ограничить число позиций, уходящих в ``evaluate``;
- матчинг и экстракцию оставить основой решения;
- итог делать коротким: “подходит / не подходит / спорно”.

Для ``equipment_deep``:

- разрешить больше спорных пар на смысловую оценку;
- добавлять объяснение причин;
- включать секции “риски”, “неполные соответствия”, “что нужно уточнить”.

И очень важно: ``equipment`` не надо сводить к одному из двух существующих классов задач.

- Это не просто ``document_analysis``, потому что там цель не “понять документ”, а “сопоставить позиции”.
- Это не просто ``compare_documents``, потому что здесь есть особая предметная логика: извлечение товарных позиций, нормализация, matching, соответствие требованиям.

Поэтому в будущем MCP-слое я бы делал отдельные slash-команды:

- ``/ask-doc``
- ``/doc-fast``
- ``/doc-deep``
- ``/compare-fast``
- ``/compare-deep``
- ``/equipment-fast``
- ``/equipment-deep``

Такой набор лучше согласуется и с MCP-подходом к отдельным tools, и с общими практиками построения инструментальных систем: чем яснее назван инструмент и его границы, тем проще и модели, и пользователю, и телеметрии. OpenAI в рекомендациях по построению инструментальных сценариев тоже делает упор на ясные инструкции и предсказуемый формат работы, а не на один чрезмерно “умный” скрытый маршрут.

Если собрать итог в одну фразу: да, ``equipment`` нужно включать в тот же вектор развития, но не как придаток compare, а как третью специализированную ветку с собственными fast/deep-режимами. Это будет и технически чище, и продуктивно понятнее, и лучше подготовит вас к переходу на MCP/slash без зависимости от большого оркестратора.

Пользователь:

И в плане реализации это всё должно выходить в open web ui, вместо chainlit, хоть сейчас у меня пока что реализация на chainlit стоит.

Ассистент:

Да — тогда целевой контур нужно строить уже не вокруг Chainlit, а вокруг Open WebUI как пользовательского интерфейса, при том что ваш текущий FastAPI/agent backend остаётся “мозгом” системы. По вашему коду это даже выглядит естественно: ``chainlit_app.py`` сейчас в основном держит состояние сессии, загрузку файлов, выбор режима и прогресс исполнения, а ``agent_api.py`` уже оформлен как отдельный API-слой с ``OrchestrationRequest``, control-plane полями и даже с явной дедупликацией запросов от Open WebUI.

Я бы формулировал это так:

Chainlit у вас сейчас — временная оболочка.
Open WebUI должен стать основной оболочкой.
FastAPI + ваши workflow — стабильное backend-ядро.

Это хорошо совпадает и с документацией Open WebUI: у него есть встроенная поддержка MCP начиная с v0.6.31, а для большинства production-сценариев сами авторы советуют рассматривать OpenAPI как основной интеграционный путь, а MCP — как удобный протокол инструментов там, где он действительно нужен. Они прямо пишут: для большинства развёртываний OpenAPI остаётся предпочтительным путём, а многие команды держат OpenAPI внутри и MCP на краю для конкретных клиентов.

Самая практичная схема для вас выглядит так.

Что оставить основой

Не переносить бизнес-логику в Open WebUI.
Логике выбора режима, исполнения и анализа лучше оставить в вашем backend.

Почему:

- у вас уже есть нормальный API-контракт в ``agent_api.py``;

- ``document_analysis``, ``compare`` и ``equipment`` уже оформлены как отдельные workflow, и это правильнее развивать на backend-стороне, а не в UI;
- Open WebUI сам по себе позиционируется как модульная оболочка, которая подключает внешние инструменты и стандартные API, а не как место, куда стоит зашивать предметную логику приложения.

То есть целевая архитектура:

Open WebUI -> ваши инструменты / API -> ваши workflow / сервисы.

Как именно это разложить

Я бы делал двухслойную интеграцию.

1) Базовый рабочий слой – OpenAPI / HTTP API

Сначала дать Open WebUI стабильные backend-эндпоинты для ваших сценариев:

- ``ask_document``
- ``analyze_document_fast``
- ``analyze_document_deep``
- ``compare_documents_fast``
- ``compare_documents_deep``
- ``analyze_equipment_fast``
- ``analyze_equipment_deep``

Это лучше для первого этапа, потому что Open WebUI умеет подключать OpenAPI-серверы как tools, а документация прямо отмечает преимущества OpenAPI: устойчивость, стандартные HTTP-коды, идемпотентность, трассировка и политики доступа.

Для вас это особенно удобно, потому что ``agent_api.py`` уже выглядит как заготовка такого слоя: там есть единый запрос, режимы, история, активные документы, control-plane и forced route. То есть вам не нужно выдумывать новый backend – он уже почти есть.

2) Целевой слой – MCP tools

После этого поверх тех же сценариев добавить MCP-сервер с отдельными инструментами.

Это правильно по двум причинам:

- MCP по спецификации предполагает отдельно именованные инструменты со схемой, а не один “магический” роутер;
- Open WebUI умеет подключаться к MCP по Streamable HTTP, а для не-HTTP вариантов предлагает MCP0-прокси.

То есть ваша зрелая схема должна быть такой:

- внутри: workflow и сервисы;
- снаружи: либо OpenAPI tools, либо MCP tools;
- в UI: Open WebUI.

Что делать с Chainlit

Я бы не переносил Chainlit-логику 1-в-1.

У вас в ``chainlit_app.py`` много вещей, которые специфичны именно для Chainlit:

- ``cl.on_chat_start``;
- ``cl.Step``;
- локальное хранение состояния в ``cl.user_session``;
- собственный data layer;
- ручная отрисовка прогресса и control plane внутри Chainlit UI.

В Open WebUI это не надо переносить буквально. Нужно вынести из Chainlit только полезные обязанности:

- хранение run/session metadata – в backend state store;
- progress / status – как данные API или MCP;
- загрузку файлов – через Open WebUI attachments + ваш document server;
- выбор режима – через отдельные tools или через явный параметр.

Иначе вы получите “Chainlit внутри Open WebUI”, а это лишний слой.

Как я бы поменял роли компонентов

Было сейчас

- Chainlit — и UI, и состояние, и часть orchestration UX.
- FastAPI backend — выполнение.
- Workflow — анализ.

Должно стать

- Open WebUI — только UI, подключение моделей, tools, файлов, доступов.
- FastAPI backend — API-шлюз и совместимый слой.
- Workflow — исполнение сценариев.
- MCP/OpenAPI layer — внешний контракт инструментов.

И вот это уже очень хорошо бьётся с тем, что Open WebUI сам строится вокруг стандартных интерфейсов: OpenAI-compatible chat API, OpenAPI-инструментов, MCP и плагинной модели расширения.

Где здесь место оркестратору

С учётом Open WebUI я бы сразу понизил роль оркестратора.

Не “убрать немедленно”, а сделать так:

- manual mode — Open WebUI вызывает конкретный tool;
- compat mode — старый оркестратор ещё жив, но уже выбирает не внутренние ветки, а внешне оформленные сценарии;
- off — оркестратор отключён, пользователь или UI выбирает tool явно.

Это особенно логично, потому что MCP по своей модели не требует, чтобы весь выбор делал скрытый оркестратор. В протоколе tools — это discoverable операции, а конкретная UX-модель остаётся на стороне клиента.

Что делать с вашим fast/deep и equipment в Open WebUI

Вот здесь я бы не усложнял.

В Open WebUI это должно выглядеть как отдельные пользовательские инструменты, а не как один универсальный prompt.

То есть наружу:

- `ask\_document`
- `analyze\_document\_fast`
- `analyze\_document\_deep`
- `compare\_documents\_fast`
- `compare\_documents\_deep`
- `analyze\_equipment\_fast`
- `analyze\_equipment\_deep`

Это согласуется и с вашим кодом, где `document\_analysis`, `compare` и `equipment` уже разделены как самостоятельные workflow. `equipment` у вас вообще отдельный предметный конвейер: extract -> route -> match -> evaluate -> report, с семантическим матчингом через Legal Server и пакетной LLM-оценкой, так что его точно не надо прятать как подвариант compare.

Самый правильный путь миграции

Я бы делал в 4 этапа.

Этап 1. Backend-first

Сначала стабилизировать backend-контракты:

- вынести отдельные endpoint/tool contracts;
- добавить `depth`;
- добавить `selection\_mode`;
- не держать эту логику только в Chainlit.

Этап 2. Open WebUI как новый фронт

Подключить Open WebUI к вашему backend:

- модели — как обычно через OpenAI-compatible путь;
- ваши сценарии — как OpenAPI tools;
- файлы — через штатную загрузку Open WebUI плюс ваш document server.

Этап 3. MCP как целевой инструментальный интерфейс

Когда contracts устоятся, те же сценарии поднять как MCP tools по HTTP.

Open WebUI умеет работать с MCP Streamable HTTP и отдельно подчёркивает, что нужен постоянный `WEBUI\_SECRET\_KEY`, иначе OAuth и сохранённые токены для MCP будут ломаться после рестарта. Это важно сразу учитывать в продовой схеме.

Этап 4. Выключение Chainlit

Когда все основные сценарии доступны через Open WebUI, Chainlit оставить только как dev/test интерфейс или убрать совсем.

Практический вывод

Если перевести на простой русский:

- не надо переносить Chainlit в Open WebUI как есть;
- надо вынести ваши сценарии в отдельные backend-инструменты;
- Open WebUI сделать главным интерфейсом;
- Chainlit использовать только как временную или отладочную оболочку;
- OpenAPI взять как первый стабильный внешний контракт, а MCP — как целевой инструментальный слой.

Самая здравая формула для вас сейчас:

FastAPI/agent_api — ядро.
Open WebUI — основной интерфейс.
Chainlit — временный переходный UI.
MCP — целевой протокол инструментов.
OpenAPI — практичный первый шаг.

И да: по вашему коду это не “перепридумать всё с нуля”, а скорее переставить роли компонентов. `agent\_api.py` уже явно готовится к сценарию Open WebUI, а Chainlit сейчас просто закрывает UX-дыры, которые потом лучше вынести в API и tools.

Могу следующим сообщением собрать это уже в виде конкретного плана миграции на Open WebUI по шагам: какие endpoints/tools завести, что оставить в backend, а что выбросить из Chainlit.

Пользователь:

Пока что по оркестратору действительно вопрос. Он у нас выступает в роли классификатора, модель Qwen3-Embedding-0.6B. И как сделать эту модульность? То есть её настроить

Ассистент:

Да — и здесь я бы развёл два понятия:

1. оркестратор как “сердце всей логики”;
2. классификатор как один из модулей выбора маршрута.

У вас сейчас, судя по коду, оркестратор уже ближе ко второму варианту: он принимает `classifier\_result`, умеет работать с `forced\_route`, учитывает `runtime\_mode`, число документов, сигналы compare/equipment/doc-question и уже потом выбирает исполнителя. А в `execution\_runtime.py` у вас вообще есть заготовка под гибридный классификатор: embedding-классификатор, LLM-классификатор и `select\_classifier\_result(...)` поверх них. То есть модульность у вас не надо “изобретать”, её надо доделать и оформить явно.

Самый важный вывод такой:

Qwen3-Embedding-0.6B нормально подходит как быстрый классификатор намерения, но его не стоит считать самым оркестратором.

Почему: Qwen3-Embedding серия сама позиционируется как модели для embeddings, retrieval, reranking, text classification и clustering, то есть для быстрого смыслового распознавания она подходит. Но это всё равно инструмент распознавания, а не слой политики исполнения.

Как я бы это разложил по-модульному

Не “один оркестратор”, а 4 отдельных слоя.

Первый слой — распознавание запроса

Он отвечает только на вопрос: “на что это больше похоже?”

Выход у него простой:

- `document\_question`
- `document\_analysis`
- `compare\_documents`
- `equipment\_analysis`

- `general\_chat`
- `greeting`
- `unsure`

Здесь как раз и живёт ваш Qwen3-Embedding-0.6B.

То есть это не оркестратор, а модуль семантического роутинга первого шага.

Второй слой – правила и сигналы

Он проверяет то, что embeddings сами по себе не знают:

- сколько документов загружено;
- есть ли 2 документа;
- есть ли признаки equipment;
- есть ли явный slash/manual override;
- есть ли forced route;
- какой сейчас режим: `auto`, `chat\_only`, потом `manual`, `compat` и так далее.

Именно этот слой должен решать:

“даже если классификатор думает X, можем ли мы вообще запускать X?”

Третий слой – выбор инструмента

Вот здесь уже появляется не “интент вообще”, а конкретный исполняемый путь:

- `ask\_document`
- `analyze\_document\_fast`
- `analyze\_document\_deep`
- `compare\_documents\_fast`
- `compare\_documents\_deep`
- `analyze\_equipment\_fast`
- `analyze\_equipment\_deep`

То есть это уже не NLP-классификация, а policy layer.

Четвёртый слой – исполнение

Это сами workflow, которые уже ничего не “угадывают”, а просто выполняют выбранный сценарий.

Что это даёт practically

Тогда модульность настраивается не через “включить/выключить оркестратор целиком”, а через заменяемые куски.

Я бы сделал такие переключатели.

1. Модуль классификации

routing:

classifier:

mode: embedding # embedding | llm | hybrid | off

embedder_model: Qwen3-Embedding-0.6B

llm_model: ...

low_confidence_threshold: 0.55

low_margin_threshold: 0.08

Смысл:

- `embedding` – только быстрый классификатор;
- `llm` – только LLM-классификатор;
- `hybrid` – embeddings сначала, LLM только при неуверенности;
- `off` – вообще без классификатора, только manual/slash или rule-based.

Это у вас очень хорошо ложится на текущую структуру `INTENT\_CLASSIFIER\_MODE` и `\_resolve\_classifier\_result\_for\_request()`.

2. Модуль policy

routing:

policy:

mode: rules # rules | rules_plus_llm | manual_only

allow_forced_route: true

ask_user_on_low_confidence: true
specialized_fallback: true

Смысл:

- `rules` – решение только по правилам;
- `rules\_plus\_llm` – если всё неоднозначно, policy может спросить маленькую модель;
- `manual\_only` – никакой автоматический выбор, только slash/tool.

3. Модуль tool selection

routing:

tools:

default_depth: fast

allow_auto_depth_upgrade: true

compare_semantic_only_in_deep: true

equipment_llm_in_fast: limited

Вот это и есть настоящая модульность исполнения.

4. Режим работы всей системы

orchestrator:

mode: compat # enabled | compat | off

Я бы закладывал именно такие значения:

- `enabled` – текущее поведение, оркестратор сам выбирает почти всё;
- `compat` – оркестратор только собирает сигналы и выбирает уже оформленный tool;
- `off` – никакого авто-роутинга, Open WebUI вызывает конкретный tool напрямую.

Какая схема будет самой здоровой именно у вас

С учётом того, что вы хотите уйти в Open WebUI и MCP, я бы советовал не усиливать оркестратор как монолит, а сделать его тонким настраиваемым слоем.

То есть:

- Qwen3-Embedding-0.6B оставить как быстрый классификатор первого шага;
- policy вынести в отдельный модуль правил;
- исполнение вынести в отдельные tools;
- оркестратор свести к клею между этими частями.

Это особенно хорошо совпадает с MCP и Open WebUI: Open WebUI умеет работать и с MCP, и с OpenAPI-инструментами, а MCP по своей модели предполагает явно именованные tools со схемой, а не один скрытый супер-маршрутизатор. Для production-сценариев Open WebUI отдельно отмечает, что OpenAPI часто остаётся самым практичным базовым путём, а MCP удобно добавлять как инструментальный слой.

Где именно поставить границы модулей

Я бы провёл их так.

Модуль A. `intent\_classifier`

Вход:

- текст запроса

Выход:

- intent
- confidence
- margin
- abstained / unsure

Никакой информации о документах, режимах и UI он не знает.

Модуль B. `routing\_signals`

Вход:

- query

- file_count
- session_docs
- active_doc_ids
- forced_route
- runtime_mode
- slash command

Выход:

- структурированные сигналы:
 - `two\_docs`
 - `has\_any\_docs`
 - `equipment\_signal`
 - `compare\_signal`
 - `manual\_override`
 - `kb\_scope`
 - и так далее

Модуль C. `route\_policy`

Вход:

- classifier_result
- routing_signals

Выход:

- `route`
- `depth`
- `selection\_mode`
- `reason`

Вот это и есть место, где принимается решение.

Модуль D. `tool\_executor`

Вход:

- уже готовый `route + depth`

Выход:

- запуск нужного workflow

Как настраивать это без боли

Я бы делал не через разбросанные env-переменные, а через единый конфиг маршрутизации.

Например:

routing:

runtime_mode_default: auto

classifier:

mode: hybrid

embedder_model: Qwen3-Embedding-0.6B

llm_model: intent-llm-small

low_confidence_threshold: 0.55

low_margin_threshold: 0.08

policy:

mode: rules

ask_user_on_low_confidence: true

allow_manual_override: true

force_doc_question_when_single_doc: true

force_compare_when_two_docs_and_compare_signal: true

force_equipment_when_detected: true

tool_selection:

document_analysis_default_depth: fast

compare_default_depth: fast

equipment_default_depth: fast

auto_upgrade_to_deep:

document_analysis: true
compare_documents: true
equipment_analysis: true

orchestrator:
mode: compat

Почему так лучше:

- всё поведение видно в одном месте;
- можно катать A/B-тесты;
- можно менять только policy, не трогая classifier;
- можно выключить classifier и оставить ручной режим.

Какая логика выбора была бы оптимальной

Я бы делал так:

Шаг 1. Если есть `forced\_route` или slash-команда – брать её сразу.
Шаг 2. Если `manual\_only` – ничего не классифицировать.
Шаг 3. Иначе запускать embedding-классификатор.
Шаг 4. Если он не уверен – либо LLM fallback, либо вопрос пользователю, либо safe default.
Шаг 5. После route отдельно выбрать `depth`.

Это важно:

сначала выбрать тип задачи, потом глубину, а не всё сразу.

Что делать именно с Qwen3-Embedding-0.6B

Я бы оставил его в роли:

- дешёвого первичного роутера;
- возможного матчера примеров интенгов;
- возможно – сигнала для confidence/margin.

Но я бы не давал ему решать финально в таких случаях:

- запрос на границе между `document\_question` и `document\_analysis`;
- случаи с `equipment`;
- случаи, где загружено 2 документа и нужна предметная логика;
- low confidence / low margin.

То есть:

embedding model = быстрый фильтр,
policy = финальное решение.

Это ещё и методологически ближе к best practices: когда система сложная, лучше не надеяться на один “умный” промпт или одну модель, а задавать явные инструкции, явные стадии и явные критерии выбора.

Если совсем коротко: как сделать модульность

Сделайте у оркестратора не один режим “вкл/выкл”, а три независимых переключателя:

- классификатор: `embedding | llm | hybrid | off`
- policy: `rules | rules\_plus\_llm | manual\_only`
- оркестратор: `enabled | compat | off`

И ещё одно обязательное поле результата:

- `selection\_mode = auto | manual | forced | fallback`

Тогда система станет настраиваемой без переписывания ядра.

Мой практический совет

На вашем месте я бы прямо сейчас делал так:

1. Переименовал мысленно оркестратор из “классификатора” в routing policy layer.
2. Qwen3-Embedding-0.6B оставил как intent classifier, не больше.

3. Вынес policy в отдельный модуль с конфигом.
4. Добавил ``orchestrator.mode = enabled|compat|off``.
5. Добавил ``classifier.mode = embedding|llm|hybrid|off``.
6. Постепенно переводил Open WebUI на прямой вызов tools, чтобы ``compat`` со временем стал основным переходным режимом.

Итоговая формула будет такой:

не “оркестратор = модель-классификатор”,
а “оркестратор = набор правил и переключателей,
а модель — только поставщик сигнала для выбора маршрута”.

Могу следующим сообщением дать уже совсем прикладной вариант: какие классы/интерфейсы завести в коде и как именно разрезать текущий ``orchestration_runtime.py`` на 3–4 модуля без большого рефакторинга.

Пользователь:

Вот как раз части по второму и первому слою лишние. Я хотел, чтобы эту часть мы убирали как-то, перейдя к ручному выбору инструмента. А при мощных серверах можно просто добавлять эти уже инструменты как tools по своей сути агенту, который сам решает что брать для решения задач. В нашем случае при ограниченных ресурсах время работы действительно имеет существенный перевес.

Ассистент:

Да — тогда я бы осознанно убирал автоматический классификатор и ранний роутинг из критического пути. При вашем приоритете на время ответа это выглядит самым рациональным решением: пользователь вручную выбирает инструмент, а backend сразу запускает нужный workflow, без промежуточного шага “сначала понять, что хотел пользователь”. Open WebUI это поддерживает по своей архитектуре: он умеет подключать внешние инструменты через OpenAPI и MCP, то есть ваш UI может работать именно как оболочка для явного выбора tools, а не как место, где обязательно нужен скрытый оркестратор.

То есть я бы поменял базовую идею так:

не
``запрос -> классификатор -> правила -> выбор инструмента -> выполнение``

а
так
``пользователь / slash-команда -> конкретный инструмент -> выполнение``

Это особенно хорошо подходит к MCP-модели: в протоколе tools — это отдельно именованные операции со схемой аргументов, а клиент или модель уже решают, какой инструмент вызвать. Иными словами, MCP не требует, чтобы у вас обязательно был свой внутренний классификатор перед каждым запуском.

В вашем случае я бы сделал три режима системы, а не один “умный” оркестратор:

1. Manual mode — основной режим для слабых серверов

Пользователь явно вызывает:

- ``ask_document``
- ``analyze_document_fast``
- ``analyze_document_deep``
- ``compare_documents_fast``
- ``compare_documents_deep``
- ``analyze_equipment_fast``
- ``analyze_equipment_deep``

Здесь оркестратор почти не нужен: он только валидирует вход, проверяет число документов, формат и права доступа, после чего сразу запускает нужный workflow. Это даёт минимальную задержку и самое предсказуемое поведение. Open WebUI умеет подключать такие серверы инструментов как OpenAPI endpoints и как MCP servers.

2. Agent mode — дополнительный режим для мощных серверов

Здесь вы отдаёте те же инструменты агенту, и уже модель сама решает, какой tool взять. Это тоже хорошо ложится на MCP: модель видит список доступных tools и выбирает нужный инструмент по задаче. То есть “агентный выбор инструмента” можно оставить как отдельный, более дорогой режим, а не как обязательный путь для всех запросов.

3. Compat mode — переходный режим

Старый оркестратор ещё жив, но не как центральный мозг, а как совместимый слой для старого UI или старых сценариев. Это полезно только на период миграции. Open WebUI сам ориентирован на протольно-стандартные интеграции, а не на жёсткую привязку к одному внутреннему

роутеру.

Поэтому да: первый и второй слой, о которых я писал раньше, в вашем целевом варианте действительно можно убрать из обязательной схемы. Их имеет смысл оставить только как опциональный режим, а не как основу архитектуры. Иначе вы продолжите платить временем за автоматическое распознавание даже там, где пользователь и так знает, что ему нужно: “быстрый анализ”, “глубокое сравнение”, “проверка оборудования”.

Тогда модульность настраивается намного проще. Не через сложный роутер, а через переключаемые режимы вызова:

```
system:
invocation_mode: manual # manual | agent | compat
```

```
tools:
expose:
- ask_document
- analyze_document_fast
- analyze_document_deep
- compare_documents_fast
- compare_documents_deep
- analyze_equipment_fast
- analyze_equipment_deep
```

```
orchestrator:
enabled: false
compat_mode: false
```

А для мощных серверов:

```
system:
invocation_mode: agent # manual | agent | compat

agent:
tool_choice: auto
allow_multi_tool: true
max_tool_calls: 3
```

И вот это уже очень близко к вашей цели:
на слабых серверах — ручной выбор и минимальная задержка;
на сильных — те же инструменты, но модель сама выбирает маршрут.

Отдельно важный момент про ваш текущий классификатор на `Qwen3-Embedding-0.6B`: сама модельная серия Qwen3 Embedding предназначена для embeddings, retrieval, ranking и связанных задач, то есть использовать её как лёгкий классификационный сигнал можно, но это скорее компромиссный быстрый роутер, а не то, что обязательно должно стоять перед каждым пользовательским запросом. Если вы уходите в manual-first, этот слой вполне можно выключить из основного пути и оставить только для agent mode или для старой совместимости.

Если перевести всё в совсем практичное решение, я бы рекомендовал такую целевую схему:

- по умолчанию: manual-first, без классификатора;
- в Open WebUI: отдельные tools и slash-команды;
- в backend: только проверка аргументов, запуск нужного workflow и возврат результата;
- для мощной конфигурации: включаем agent mode, где эти же tools доступны модели для автоматического выбора;
- оркестратор: превращаем в опциональный compatibility layer, а не в обязательный узел системы.

Итоговая формула для вас, по-моему, такая:

manual-first для экономии времени и ресурсов;
agent-with-tools как premium-режим для мощных серверов;
оркестратор — не ядро, а отключаемая прослойка.

Следующим сообщением я могу расписать это уже как конкретную конфигурационную схему: какие флаги ввести (`manual`, `agent`, `compat`), какие tools зарегистрировать в Open WebUI и как

переупростить текущий `orchestration\_runtime.py` под такой режим.

Пользователь:

А как эти mods настраивать, например в open web ui, там есть такие вкладки? Но лучше его вовсе убрать наверное

Ассистент:

Да — в Open WebUI нет готовых вкладок именно под ваши `manual / agent / compat` режимы. Это не встроенные режимы продукта. Их нужно задавать в вашей системе, а Open WebUI использовать как оболочку: подключить tools, выдать доступы и дать пользователю выбрать нужный инструмент. В Open WebUI для этого есть два основных места: Workspace -> Tools для пользовательских/рабочих tools и Admin Settings -> Tools для общих tool servers; отдельно Functions живут не там же, а в Admin Panel, потому что это уже расширение поведения самой платформы, а не просто внешний инструмент.

Поэтому ваш вывод “лучше оркестратор убрать” в вашем сценарии звучит разумно. Если приоритет — минимальная задержка и ограниченные ресурсы, то самый дешёвый путь такой:

Open WebUI -> явный выбор tool -> ваш backend endpoint / MCP tool -> workflow.

Без промежуточного:

запрос -> классификатор -> роутер -> tool.

Это хорошо совпадает и с архитектурой Open WebUI: он умеет подключать внешние инструменты через OpenAPI tool servers и через MCP, а для большинства развёртываний сами авторы отдельно отмечают, что OpenAPI часто остаётся самым практичным базовым путём, а MCP можно добавлять там, где он реально нужен.

Как я бы делал у вас.

Что настраивать в Open WebUI

Не “моды” в смысле внутренних режимов, а список подключённых инструментов.

То есть в Open WebUI вы настраиваете не:

- `manual`
- `agent`
- `compat`

а подключаете, например:

- `ask\_document`
- `analyze\_document\_fast`
- `analyze\_document\_deep`
- `compare\_documents\_fast`
- `compare\_documents\_deep`
- `analyze\_equipment\_fast`
- `analyze\_equipment\_deep`

И уже это становится вашим ручным выбором.

Где это живёт:

- для общих инструментов на весь инстанс — Admin Settings -> Tools;
- для workspace-уровня — Workspace Tools; при этом они выполняют Python-код на сервере, и документация прямо предупреждает, что это чувствительный по безопасности механизм.

Что тогда делать с вашими режимами

Я бы их вообще не тащил в UI как отдельные переключатели, если вы всё равно хотите убрать оркестратор.

Вместо этого:

- `manual` = просто пользователь явно вызывает нужный tool;
- `agent` = отдельная конфигурация, где модели разрешён автоматический tool choice;
- `compat` = временный backend-режим для старого клиента, но не часть постоянного UI.

То есть в Open WebUI вы не настраиваете “режимы оркестратора”.

Вы либо:

- показываете пользователю набор tools,
- либо даёте модели доступ к этим tools.

Это проще и дешевле.

Что лучше для вас прямо сейчас

С вашим ограничением по времени ответа я бы выбрал такой путь:

1. Оркестратор убрать из основного пути.

Оставить только тонкую проверку:

- хватает ли документов;
- подходит ли тип входа;
- валидны ли аргументы.

2. В Open WebUI подключить backend как tools.

Сначала проще всего через OpenAPI tool server. Open WebUI умеет ingest OpenAPI-спецификацию и трактовать endpoints как tools.

3. MCP добавить потом, если понадобится.

Нативная поддержка MCP в Open WebUI есть с версии v0.6.31+, а для non-HTTP/stdio-сценариев у них есть MCP0-прокси.

То есть отвечая прямо на ваш вопрос

Есть ли в Open WebUI вкладки под такие “mods”?

Нет, не в том виде, как вы это описываете. Есть вкладки и разделы для:

- Tools,
- Functions,
- Integrations,
- Models,

но не готовая система ваших кастомных режимов `manual / compat / agent`. Это уже ваша логика поверх Open WebUI.

Лучше ли убрать оркестратор?

Для вашего текущего профиля – скорее да. Не обязательно удалить код в ноль, но точно:

- убрать его из обязательного критического пути;
- не гонять каждый запрос через классификатор;
- оставить, максимум, как временный совместимый слой.

Самая практичная целевая схема

Я бы делал так:

- Open WebUI – основной интерфейс;
- ваш FastAPI backend – tool server;
- каждый workflow – отдельный tool;
- ручной выбор инструмента – по умолчанию;
- автоматический выбор агентом – только на мощных серверах и как отдельная опция;
- оркестратор – либо выключен, либо только для совместимости.

Это будет и быстрее, и проще поддерживать.

Если хотите, я могу следующим сообщением дать вам совсем конкретную карту по Open WebUI: в какие разделы зайти, что именно там создать, и как разложить ваши 7 инструментов без оркестратора.

Пользователь:

Итак, я это пожалуй учту, но пока что будет двигаться по сценарию без профилей

Ассистент:

Да, это разумно.

Для вашего текущего этапа вариант “без профилей” выглядит самым практичным: меньше лишней конфигурации, меньше ветвления в UI и меньше скрытой логики, которая всё равно потом будет мешать отладке. Open WebUI и так даёт базовые точки расширения через Tools, OpenAPI Tool Servers и MCP, так что сначала можно просто выставить отдельные инструменты и не вводить поверх них ещё один слой “профилей” или режимов.

Я бы зафиксировал для себя такую временную цель:

- без профилей
- без обязательного оркестратора
- с явными tools
- с ручным выбором инструмента как основным сценарием

То есть пользователь выбирает не “режим”, а сразу действие:

`ask\_document`, `analyze\_document\_fast`, `analyze\_document\_deep`, `compare\_documents\_fast`, `compare\_documents\_deep`, `analyze\_equipment\_fast`, `analyze\_equipment\_deep`.

Это хорошо совпадает и с MCP-подходом: инструменты должны быть отдельно именованными

операциями с понятной схемой входа, а клиент или модель уже решают, какой именно инструмент вызвать.

Если говорить совсем прикладно, то ваш ближайший путь я бы видел так:

1. Оставить backend как набор отдельных сценариев.
2. Подключить их в Open WebUI как tools.
3. Убрать “профили” из проектного плана до тех пор, пока не появится реальная боль от ручного выбора.
4. Оркестратор держать только как временный совместимый слой или вообще не использовать в основном пути.
5. Автоматический выбор инструмента вернуть потом, только если появятся более мощные серверы и станет выгодно давать модели самой выбирать tool.

Ещё один важный момент: если вы пойдёте через Workspace Tools, Open WebUI прямо предупреждает, что такие инструменты исполняют произвольный Python-код на сервере, поэтому для вашей архитектуры безопаснее и чище делать ставку именно на внешний backend через OpenAPI или MCP, а не зашивать логику внутрь самого WebUI.

Итог у вас сейчас вполне здоровый:

сначала простая схема без профилей и без сложного роутинга, потом – при необходимости – агентный выбор tools.

Это как раз тот случай, где лучше не переусложнять раньше времени. Когда дойдёте до следующего шага, уже можно будет отдельно решить, что выгоднее: оставить только ручной выбор или добавить поверх него agent mode.

Пользователь:

Итак, на основе всей нашей с тобой переписки сделай мне конкретный подробный технический план в виде чекбоксов (+ контекст с объяснением), то есть как тз для разработчика по реализации и переходу к open web ui и всей этой полноценной реализацией под то, что у нас сейчас имеется в chainlit и по роутингу, чтобы переход был плавный.

Формат задачи в md.

Ассистент:

Техническое задание: плавный переход с Chainlit на Open WebUI без обязательного оркестратора

1. Цель

Перевести текущую систему с `Chainlit` на `Open WebUI` так, чтобы:

- основным пользовательским интерфейсом стал **Open WebUI**;
- текущий backend на **FastAPI** остался основным исполняющим слоем;
- сценарии ``document_question``, ``document_analysis``, ``compare_documents``, ``equipment_analysis`` остались рабочими;
- переход был **плавным**, без одномоментного удаления текущего кода;
- **автоматический роутинг и классификатор были убраны из критического пути**;
- основным режимом стал **ручной выбор инструмента**, потому что для текущей инфраструктуры время ответа и экономия ресурсов важнее, чем “умный” автовыбор. Open WebUI умеет подключать внешние инструменты через **OpenAPI Tool Servers** и через **MCP**, поэтому такая архитектура ему подходит.

2. Контекст текущего состояния

Сейчас у вас уже есть хорошая основа для миграции:

- ``chainlit_app.py`` используется как UI-оболочка, в нём есть хранение сессии, шаги прогресса ``cl.Step``, локальная persistence-логика и восстановление состояния/документов; это значит, что Chainlit сейчас выполняет прежде всего **UI- и session-роль**, а не является центром доменной логики.
- ``agent_api.py`` уже оформлен как отдельный FastAPI-слой с ``OrchestrationRequest``, OpenAI-совместимым API, ``/orchestrate``, ``/execute_orchestration``, ``/v1/chat/completions``, а также дедупликацией параллельных запросов от Open WebUI; значит, backend уже близок к роли внешнего tool server.
- ``execution_runtime.py`` всё ещё тянет в себя intent-классификацию (``EmbeddingIntentClassifier``, ``LLMIntentClassifier``, ``select_classifier_result``) и runtime-политику; это как раз тот слой, который нужно перестать считать обязательным перед каждым вызовом.

- `document\_analysis.py` уже построен как многошаговый workflow с map-reduce-суммаризацией и budget-aware логикой; `summary\_reduce\_policy.py` отдельно держит логику `fast\_final\_merge` и правила деградации. Это хорошая база для fast/deep-исполнения без второго “умного оркестратора”.

- `compare.py` уже фактически разделяет структурное сравнение и более глубокий смысловой fallback; значит, тут нужно не придумывать новый алгоритм, а сделать явные tool-контракты.

- `equipment.py` уже является отдельным специализированным workflow: двухпроходная экстракция, short-circuit на таблицах, LLM-оценка пар, отдельный граф `extract -> match -> evaluate -> report`; его надо переносить как отдельный инструмент, а не как подвид compare.

3. Целевое архитектурное решение

Базовый принцип

****Не переносить Chainlit-логику “как есть” в Open WebUI.****

Нужно вынести наружу ****явные backend-инструменты****, а Open WebUI использовать как интерфейс, который умеет вызывать такие инструменты. Open WebUI для этого поддерживает OpenAPI Tool Servers и MCP, а Tools и Functions у него разведены по разным разделам платформы.

Целевая схема

`Open WebUI -> внешний tool server (FastAPI backend) -> workflow`

Ключевое решение по роутингу

****Автоматический классификатор и обязательный оркестратор убрать из основного сценария.****
Основной сценарий:

`пользователь / UI выбирает конкретный tool -> backend сразу исполняет его`

Это согласуется и с MCP: tools – это отдельно именованные операции со схемой входа, а клиент или модель уже выбирают, какой вызвать. MCP не требует, чтобы перед каждым tool-вызовом работал отдельный внутренний классификатор.

4. Что должно стать отдельными инструментами

Обязательный минимальный набор tools

- [] `ask\_document`
- [] `analyze\_document\_fast`
- [] `analyze\_document\_deep`
- [] `compare\_documents\_fast`
- [] `compare\_documents\_deep`
- [] `analyze\_equipment\_fast`
- [] `analyze\_equipment\_deep`

Контекст

Это лучше, чем один “универсальный” endpoint с внутренним скрытым выбором, потому что:

- Open WebUI лучше работает с явно подключёнными инструментами;
- MCP именно так и моделирует инструменты – отдельно именованные действия с предсказуемой схемой;
- ваша текущая кодовая база уже логически разделена на самостоятельные workflow по этим сценариям.

5. Что делать с оркестратором

Целевое решение

- [] Убрать оркестратор из обязательного пути исполнения.
- [] Оставить его только как ****временный compatibility-layer**** на переходный период.
- [] После миграции на Open WebUI перевести оркестратор в статус ****опционального legacy-модуля****.
- [] Не запускать `EmbeddingIntentClassifier` / `LLMIntentClassifier` перед каждым пользовательским запросом.
- [] Исключить обязательный шаг `decide\_orchestration(...)` из manual-first сценария.

Контекст

Сейчас оркестратор и execution-слой всё ещё завязаны на классификатор, `forced\_route`, `runtime\_mode`, pending action и control-plane-логику. Это удобно для Chainlit, но создаёт лишнюю задержку и сложность, если пользователь и так может выбрать правильный инструмент руками.

6. Принцип миграции: manual-first

Решение

- [] Зафиксировать **manual-first** как основной режим на текущей инфраструктуре.
- [] Не вводить отдельные “профили” в первой фазе.
- [] Не добавлять agent mode в обязательный MVP перехода.
- [] Автоматический выбор tools оставить только как **будущую** опцию для более мощных серверов.

Контекст

Это соответствует вашим ограничениям: сейчас время ответа и экономия ресурсов важнее, чем агентный комфорт. Open WebUI умеет работать как оболочка для инструментов и не требует обязательно строить поверх них сложную внутреннюю логику выбора.

7. План реализации backend-слоя

Этап А. Зафиксировать целевой API-контракт

- [] Создать отдельный backend-спец документ `docs/openwebui\_tools\_contract.md`.
- [] Для каждого инструмента описать:
 - [] имя tool;
 - [] входные аргументы;
 - [] обязательность файлов;
 - [] формат результата;
 - [] ошибки валидации;
 - [] требования к цитированию / groundedness;
 - [] возможность стриминга;
 - [] формат execution metadata.
- [] Зафиксировать, что `ask\_document` — это **строгое grounded QA по документу**, а не “полный анализ”.
- [] Зафиксировать, что `equipment` — отдельная предметная ветка, не подветка compare.

Минимальная целевая схема аргументов

- `ask\_document`
 - `message`
 - `active\_doc\_ids[]`
 - `session\_docs`
 - `history`
- `analyze\_document\_fast|deep`
 - `document\_id` или `path`
 - `history` (опционально)
- `compare\_documents\_fast|deep`
 - `doc\_id\_1`
 - `doc\_id\_2`
 - `history` (опционально)
- `analyze\_equipment\_fast|deep`
 - `doc\_id\_1`
 - `doc\_id\_2`
 - `mode` (опционально, если не вычисляется автоматически)
 - `history` (опционально)

Контекст

Сейчас `agent\_api.py` уже имеет общий `OrchestrationRequest`, но он перегружен control-plane и legacy-полями вроде `runtime\_mode`, `assistant\_mode`, `tool\_scope`, `classifier\_result`, `forced\_route`. Для Open WebUI лучше иметь более узкие и явные контракты на каждый инструмент.

Этап В. Отделить execution от orchestration

- [] Вынести вызовы workflow в отдельный модуль, например `tool\_executor.py`.
- [] Сделать в этом модуле ****прямые функции вызова****:
 - [] `run\_ask\_document(...)`
 - [] `run\_analyze\_document\_fast(...)`
 - [] `run\_analyze\_document\_deep(...)`
 - [] `run\_compare\_documents\_fast(...)`
 - [] `run\_compare\_documents\_deep(...)`
 - [] `run\_equipment\_fast(...)`
 - [] `run\_equipment\_deep(...)`
- [] Убедиться, что эти функции не требуют предварительного запуска intent-классификатора.
- [] Разделить:
 - [] валидацию входа;
 - [] подготовку runtime_context;
 - [] исполнение workflow;
 - [] рендер результата.

Контекст

Сейчас `execute\_orchestration` и связанные с ним слои смешивают runtime-политику, классификацию и исполнение. Для ручных tools это нужно упростить: UI уже выбрал инструмент, backend должен только проверить вход и выполнить его.

Этап С. Упростить `agent\_api.py`

- [] Не удалять `agent\_api.py`, а сделать его главным внешним server endpoint.
- [] Добавить отдельные endpoints под tools.
- [] Сохранить существующие `/health`, `/metrics`, OpenAI-compatible routes.
- [] Перенести `/orchestrate` в legacy-раздел.
- [] Явно пометить `/orchestrate` как deprecated для UI-маршрута.
- [] Оставить `/execute\_orchestration` только на переходный период.
- [] Ввести новые endpoints, например:
 - [] `POST /tools/ask\_document`
 - [] `POST /tools/analyze\_document\_fast`
 - [] `POST /tools/analyze\_document\_deep`
 - [] `POST /tools/compare\_documents\_fast`
 - [] `POST /tools/compare\_documents\_deep`
 - [] `POST /tools/analyze\_equipment\_fast`
 - [] `POST /tools/analyze\_equipment\_deep`

Контекст

Open WebUI v0.6+ поддерживает подключение внешних OpenAPI servers как tool servers, поэтому этот слой лучше сделать стабильным и явным.

Этап D. Оставить OpenAI-compatible API как совместимый слой

- [] Не ломать `/v1/chat/completions`.
- [] Оставить OpenAI-compatible API для внешних клиентов и отладки.
- [] Сохранить дедупликацию параллельных запросов.
- [] Сохранить trace-id и observability middleware.
- [] Сохранить возможность стриминга ответов.

Контекст

У вас уже есть OpenAI-compatible слой, плюс дедупликация запросов от Open WebUI. Это полезно оставить как совместимость и для безопасного перехода.

8. План реализации по workflow

Этап Е. `ask\_document` оставить отдельным grounded-RAG инструментом

- [] Не смешивать `ask\_document` с `document\_analysis`.
- [] Оставить его как режим: "найти релевантные фрагменты -> собрать ответ -> проверить цитаты / доказательства".
- [] Вынести его в отдельный endpoint/tool.

- [] Сохранить deterministic fallback, если источников недостаточно.
- [] Возвращать:
 - [] `assistant\_message`
 - [] `sources`
 - [] `quality\_signals`
 - [] `execution\_metadata`

Контекст

В текущем `execution\_runtime.py` уже есть grounded path с fallback при отсутствии источников и retrieval adapter. Это правильная модель для `ask\_document`, её не нужно превращать в “анализ”.

Этап F. `document\_analysis` разделить на fast и deep

`analyze\_document\_fast`

- [] Использовать текущий workflow как основу.
- [] Задать более жёсткие лимиты summary policy:
 - [] меньше `chunk\_input\_chars`
 - [] меньше `group\_input\_chars`
 - [] меньше `final\_input\_chars`
 - [] меньше `final\_max\_tokens`
 - [] разрешён `fast\_final\_merge`
- [] Делать короткий итоговый отчёт.
- [] Не форсировать расширенный финальный синтез, если хватает частичной/короткой сводки.
- [] Возвращать execution metadata с указанием реально выбранной reduce-стратегии.

`analyze\_document\_deep`

- [] Использовать тот же workflow, но с более “дорогой” policy.
- [] Увеличить допустимый объём финальной сборки.
- [] Разрешить больше уровней reduce.
- [] Делать полный аналитический отчёт.
- [] В итог включать больше секций и метаданных.

Контекст

`document\_analysis.py` уже имеет map-reduce-суммаризацию и адаптивную summary policy, а `summary\_reduce\_policy.py` уже держит логику fast path и деградации. Здесь нужен не новый алгоритм, а явная параметризация под два режима.

Этап G. `compare\_documents` разделить на fast и deep

`compare\_documents\_fast`

- [] Сделать основой **структурное сравнение**.
- [] LLM-анализ выполнять только для модификаций, реально требующих расшифровки.
- [] Semantic fallback не запускать автоматически в большинстве случаев.
- [] Делать короткий redline-подобный отчёт.

`compare\_documents\_deep`

- [] Поверх fast-пути включить semantic path.
- [] Разрешить heterogeneous / semantic fallback.
- [] Включать:
 - [] relation summary
 - [] key findings
 - [] coverage gaps
 - [] conflicts
 - [] recommended actions

Контекст

Сейчас `compare.py` уже сначала отделяет structural vs LLM-needed differences, а затем при определённых условиях включает semantic fallback. Это почти готовая основа для двух инструментов.

Этап H. `equipment` разделить на fast и deep

`analyze\_equipment\_fast`

- [] Оставить двухпроходную экстракцию.
- [] Сохранить short-circuit при хорошем table extraction.
- [] Ограничить число пар, уходящих в LLM evaluate.
- [] Делать короткий итог: подходит / не подходит / спорно.
- [] Минимизировать batch LLM usage.

`analyze\_equipment\_deep`

- [] Разрешить больше пар на этап `evaluate`.
- [] Добавить более подробные причины соответствия/несоответствия.
- [] Выводить спорные и рискованные позиции отдельно.
- [] Сохранять полный отчёт по позициям.

Контекст

`equipment.py` уже построен как отдельный граф `extract -> match -> evaluate -> report`, с условной маршрутизацией после extract и отдельным LLM evaluation batching. Это надо не прятать в compare, а переносить как самостоятельные tools.

9. План интеграции с Open WebUI

Этап I. Первый шаг – OpenAPI Tool Servers

- [] Подготовить OpenAPI-спецификацию для новых `/tools/\*` endpoints.
- [] Подключить backend как OpenAPI Tool Server в Open WebUI.
- [] Проверить доступность каждого tool из интерфейса Open WebUI.
- [] Для MVP использовать именно OpenAPI, а не сразу MCP.
- [] Не переносить бизнес-логику в Open WebUI Functions.

Контекст

Open WebUI прямо поддерживает OpenAPI servers как внешний способ интеграции tools; это самый практичный первый шаг. Tools и Functions у Open WebUI – разные сущности, причём Functions меняют поведение платформы и исполняют код на сервере, поэтому их лучше не использовать как место для вашей предметной логики.

Этап J. Второй шаг – MCP как целевой слой

- [] После стабилизации OpenAPI-интеграции добавить MCP server поверх тех же tool-контрактов.
- [] Делать MCP только через **HTTP / Streamable HTTP**, если возможно.
- [] Не смешивать OpenAPI-конфиг и MCP-конфиг в Open WebUI.
- [] При необходимости использовать MCPO только для нестандартных транспортов.

Контекст

Open WebUI поддерживает нативный MCP Streamable HTTP и отдельно предупреждает, что MCP нужно добавлять как MCP connection, а не как OpenAPI connection. Для не-HTTP MCP-серверов предлагается MCPO.

Этап K. Как именно это должно выглядеть в Open WebUI

- [] Не реализовывать в Open WebUI “режимы” вроде `manual / compat / agent`.
- [] В Open WebUI просто зарегистрировать набор tools.
- [] Пользовательский выбор делать через сам вызов нужного tool.
- [] Файлы загружать через стандартный механизм Open WebUI + ваш backend document path handling.
- [] Для админской настройки использовать разделы Tools / External Tools.
- [] Не переносить session-state-логику Chainlit внутрь Open WebUI.

Контекст

В Open WebUI нет готовых встроенных “вкладок” под ваши кастомные режимы маршрутизации; есть

инструменты, внешние tool servers, functions и модельные настройки. Значит, ваши режимы – это не UI-фича Open WebUI, а ваша backend-политика.

10. Что именно переносить из Chainlit, а что нет

Переносить

- [] логику загрузки и регистрации документов;
- [] логику восстановления активного набора документов;
- [] trace-id / request correlation;
- [] progress / status сообщения как backend metadata;
- [] RAG indexing lifecycle, если он нужен для `ask\_document`;
- [] cancellation hooks на уровне backend execution.

Контекст

В `chainlit\_app.py` уже есть работа с активными документами, кэшем RAG, восстановлением документов, прерыванием активного исполнения и синхронизацией состояния. Это полезная логика, но её надо вынести из Chainlit-специфичного слоя в backend-сервисы.

Не переносить буквально

- [] `cl.Step`
- [] `cl.user\_session`
- [] Chainlit Data Layer как основное хранилище
- [] pending route choice UI
- [] control-plane UI-переключатели Chainlit
- [] прямую зависимость frontend-логики от `\_backend\_decide\_orchestration`

Контекст

Это специфично для Chainlit и не должно становиться архитектурной зависимостью будущей системы.

11. План по состоянию и данным

Этап L. Вынести состояние из Chainlit-формата

- [] Ввести backend-структуру `SessionExecutionState`.
- [] Хранить отдельно:
 - [] `session\_id`
 - [] `thread\_id`
 - [] `active\_doc\_ids`
 - [] `session\_docs`
 - [] `last\_tool\_used`
 - [] `last\_execution\_metadata`
 - [] `pending\_cancel\_flag`
- [] Не использовать `cl.user\_session` как источник истины.
- [] Сделать backend state store независимым от конкретного UI.

Контекст

Сейчас Chainlit хранит значительную часть состояния внутри своей user-session модели. Для Open WebUI это надо заменить backend-слоем состояния.

Этап M. Вынести progress/update в backend response model

- [] Добавить в tool responses поле `progress\_events` или отдельный стрим статусов.
- [] Не полагаться на `cl.Step`.
- [] Унифицировать:
 - [] `stage`
 - [] `message`
 - [] `percent` или `position/total`
 - [] `started\_at`
 - [] `elapsed`
- [] Сделать так, чтобы Open WebUI видел обычный ответ, даже если подробный progress UI позже не будет отрисован.

Контекст

Chainlit даёт красивую модель шагов, но Open WebUI не обязан повторять её один в один. Поэтому прогресс нужно сначала сделать данными backend, а не UI-конструкцией.

12. План по конфигурации и feature flags

Этап N. Ввести явные флаги миграции

- [] Добавить `ENABLE\_LEGACY\_ORCHESTRATOR=true|false`
- [] Добавить `ENABLE\_OPENWEBUI\_TOOL\_ENDPOINTS=true|false`
- [] Добавить `ENABLE\_MCP\_SERVER=true|false`
- [] Добавить `DEFAULT\_INVOCATION\_MODE=manual`
- [] Добавить `ALLOW\_AGENT\_TOOL\_SELECTION=false` по умолчанию
- [] Добавить `ENABLE\_CHAINLIT\_UI=true|false`
- [] Добавить `CHAINLIT\_LEGACY\_COMPAT\_MODE=true|false`

Контекст

Переход должен быть плавным, а не одномоментным. Поэтому всё критичное нужно включать по флагам, чтобы можно было катить поэтапно. Это важнее, чем попытка сразу “вырезать” старую схему. Основа для runtime- и effective-settings-подхода у вас уже есть.

13. План по безопасности и надёжности

Этап O. Безопасность интеграции с Open WebUI

- [] Не переносить доменную логику в Open WebUI Workspace Functions.
- [] Подключать backend как внешний tool server.
- [] Ограничить CORS до реальных origin, когда фронт переедет на Open WebUI.
- [] Добавить auth/token между Open WebUI и backend tool server.
- [] Проверить path mapping между контейнером, Open WebUI и document server.
- [] Сохранить дедупликацию запросов от Open WebUI.

Контекст

Open WebUI Functions и некоторые tool-механизмы исполняют код на сервере и должны рассматриваться как чувствительный механизм. Для вашей архитектуры безопаснее держать логику во внешнем backend.

У вас уже есть дедупликация параллельных запросов от Open WebUI – это полезно сохранить.

14. План по тестированию

Этап P. Контрактные тесты backend tools

- [] Написать tests на каждый новый `/tools/\*` endpoint.
- [] Проверить:
 - [] корректную валидацию входа;
 - [] ошибки при отсутствии файлов;
 - [] ошибки при неправильном числе документов;
 - [] корректность grounded answer в `ask\_document`;
 - [] корректность формата отчёта в `document\_analysis`;
 - [] корректность парного сравнения в `compare`;
 - [] корректность equipment-режима.

Этап Q. Интеграционные тесты Open WebUI

- [] Проверить подключение OpenAPI tool server к Open WebUI.
- [] Проверить, что каждый tool доступен в интерфейсе.
- [] Проверить загрузку файлов и передачу attachment metadata.
- [] Проверить многодокументный сценарий.
- [] Проверить, что Open WebUI не дублирует запуск одного и того же тяжёлого workflow.

Этап R. Регрессионные тесты Chainlit

- [] На переходный период сохранить минимальный smoke-suite для Chainlit.
- [] Проверить:

- [] загрузку документов;
- [] восстановление истории;
- [] pending-choice сценарии;
- [] прерывание long-running обработки.
- [] После стабилизации Open WebUI перевести Chainlit в статус legacy/dev-only.

15. План по телеметрии

Этап S. Метрики по каждому tool

- [] Добавить отдельные latency-метрики по всем 7 tools.
- [] Логировать:
 - [] общее время;
 - [] число LLM-вызовов;
 - [] число вызовов document server;
 - [] факт деградации / fallback;
 - [] долю ответов без достаточных источников;
 - [] размер входных данных;
 - [] число обработанных чанков;
 - [] число пар в compare/equipment.
- [] Вывести в метаданные ответа:
 - [] `tool\_name`
 - [] `execution\_mode=manual`
 - [] `degraded=true|false`
 - [] `fallbacks[]`
 - [] `duration\_sec`

Контекст

У вас уже есть наблюдаемость, metrics endpoint и telemetry/runtime hooks. Их нужно не переписывать, а перевести с “маршрута оркестратора” на “маршрут конкретного инструмента”.

16. План вывода Chainlit из эксплуатации

Этап T. Переходный период

- [] Сначала держать Chainlit и Open WebUI параллельно.
- [] Новые tool endpoints делать общими для обоих интерфейсов.
- [] Chainlit перевести на вызов тех же новых backend tool endpoints, если это возможно.
- [] Не развивать новые фичи в Chainlit, кроме критических багфиксов.
- [] Зафиксировать дату прекращения активной продуктовой разработки в Chainlit.

Этап U. Финальная консервация Chainlit

- [] После стабилизации Open WebUI:
 - [] отключить Chainlit по feature flag;
 - [] оставить только dev/debug запуск;
 - [] убрать его из production deployment;
 - [] сохранить smoke tests на случай экстренного rollback.

Контекст

Сейчас Chainlit уже позиционируется в коде как “замена Open WebUI”, но по целевому вектору он должен стать временной оболочкой, а не главной точкой входа.

17. План релизов

Release 1 – backend preparation

- [] Выделены новые `/tools/\*` endpoints.
- [] Инструменты вызываются без обязательного оркестратора.
- [] Сохранён OpenAI-compatible API.
- [] Chainlit всё ещё работает.

Release 2 – Open WebUI MVP

- [] Backend подключён к Open WebUI как OpenAPI Tool Server.

- [] Все 7 tools доступны из Open WebUI.
- [] Ручной выбор инструмента – основной сценарий.
- [] Chainlit остаётся fallback UI.

Release 3 – stabilization

- [] Прогнаны интеграционные тесты.
- [] Сняты метрики latency/quality.
- [] Устранены расхождения между Chainlit и Open WebUI.
- [] Оркестратор больше не обязателен.

Release 4 – optional MCP

- [] Те же инструменты подняты как MCP tools.
- [] Open WebUI умеет работать и через OpenAPI, и через MCP.
- [] MCP используется там, где это реально даёт пользу.

Release 5 – legacy cleanup

- [] Chainlit выведен из production.
- [] `/orchestrate` помечен deprecated или выключен.
- [] Intent-classifier убран из основного пользовательского пути.
- [] Manual-first окончательно закреплён как дефолтный режим.

18. Явно не делать на первом этапе

- [] Не внедрять сложные “профили”.
- [] Не строить новый большой оркестратор.
- [] Не завязывать Open WebUI на custom Functions как на место основной логики.
- [] Не переносить `cl.Step` 1-в-1.
- [] Не начинать с MCP раньше, чем стабилизирован OpenAPI-контракт.
- [] Не обучать LoRA-роутер до того, как будет устойчивая manual-first схема.
- [] Не смешивать `ask\_document` с полным анализом документа.
- [] Не сводить `equipment` к подтипу compare.

19. Итоговое архитектурное решение

Финальное решение для реализации

- [] ****UI:**** Open WebUI
- [] ****Основной внешний контракт:**** OpenAPI Tool Server
- [] ****Целевой дополнительный контракт:**** MCP
- [] ****Основной режим:**** manual-first
- [] ****Оркестратор:**** только временная совместимость
- [] ****Chainlit:**** временный legacy UI
- [] ****Backend:**** FastAPI + существующие workflow
- [] ****Инструменты:**** 7 явных tools
- [] ****Классификатор:**** не используется в обязательном пути
- [] ****Agent tool selection:**** только будущий optional mode для более мощных серверов

20. Критерии готовности

- [] Open WebUI подключён к backend без Chainlit-зависимостей.
- [] Пользователь может вручную вызвать любой из 7 tools.
- [] `ask\_document` работает как grounded QA с цитатами / fallback.
- [] `document\_analysis\_fast/deep` различаются policy, а не отдельным дублированным кодом.
- [] `compare\_documents\_fast/deep` различаются степенью semantic анализа.
- [] `equipment\_fast/deep` вынесены как отдельные tools.
- [] Запуск tool не требует предварительного intent-классификатора.
- [] Chainlit больше не является обязательной частью пользовательского пути.
- [] Система даёт меньшую задержку, чем текущий сценарий с обязательным оркестратором.
- [] Переход можно включать по feature flags и откатывать без потери работоспособности.

Пользователь:

Сформируй pdf файл нашей переписки полностью (без сокращений)